```
# Continution of Lab-8
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, precision_score, recall_score, f1_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
```

```
df = pd.read_csv("/content/Churn_Modelling (1).csv")
```

```
df.drop(columns= ["CustomerId", "Surname", "RowNumber"], axis= 1 , inplace= True, errors='ignore')
```

```
# Dummification of the data
```

```
df =pd.get_dummies(df, columns= ["Geography", "Gender"], drop_first= True)
df.head()
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited	Geography_Germany
0	619	42	2	0.00	1	1	1	101348.88	1	False
1	608	41	1	83807.86	1	0	1	112542.58	0	False
2	502	42	8	159660.80	3	1	0	113931.57	1	False
3	699	39	1	0.00	2	0	0	93826.63	0	False
4	850	43	2	125510.82	1	1	1	79084.10	0	False

```
df = df.astype(int)
df.head()
```

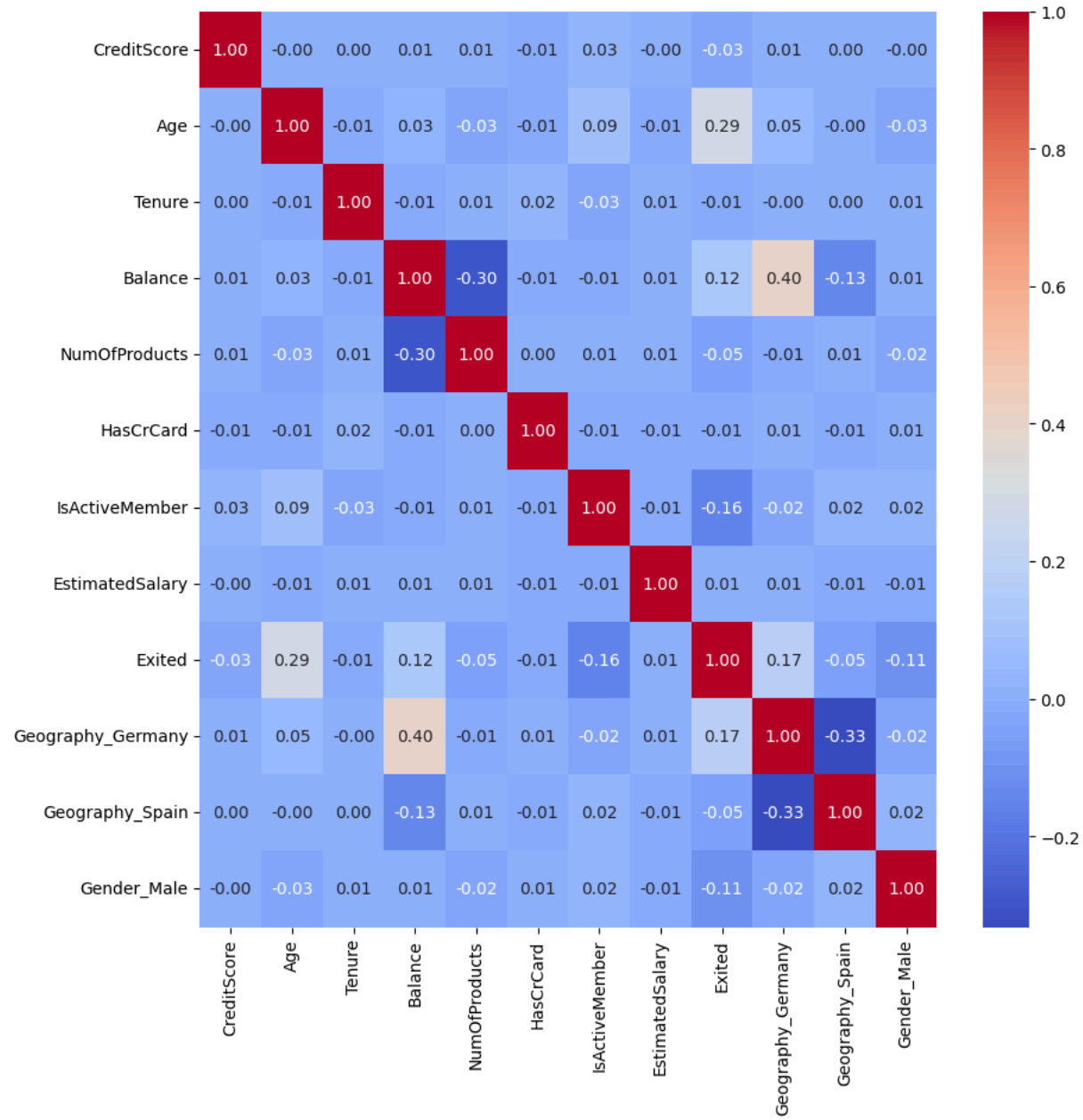
	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited	Geography_Germany	G
0	619	42	2	0	1	1	1	101348	1	0	
1	608	41	1	83807	1	0	1	112542	0	0	
2	502	42	8	159660	3	1	0	113931	1	0	
3	699	39	1	0	2	0	0	93826	0	0	
4	850	43	2	125510	1	1	1	79084	0	0	

```
# Ploting the Heat map
```

```
corr = df.corr()
corr
```

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	
CreditScore	1.000000	-0.003965	0.000842	0.006268	0.012238	-0.005458	0.025651	-0.001384	-(
Age	-0.003965	1.000000	-0.009997	0.028308	-0.030680	-0.011721	0.085472	-0.007201	(
Tenure	0.000842	-0.009997	1.000000	-0.012254	0.013444	0.022583	-0.028362	0.007784	-(
Balance	0.006268	0.028308	-0.012254	1.000000	-0.304179	-0.014858	-0.010084	0.012798	(
NumOfProducts	0.012238	-0.030680	0.013444	-0.304179	1.000000	0.003183	0.009612	0.014204	-(
HasCrCard	-0.005458	-0.011721	0.022583	-0.014858	0.003183	1.000000	-0.011866	-0.009933	-(
IsActiveMember	0.025651	0.085472	-0.028362	-0.010084	0.009612	-0.011866	1.000000	-0.011421	-(
EstimatedSalary	-0.001384	-0.007201	0.007784	0.012798	0.014204	-0.009933	-0.011421	1.000000	(
Exited	-0.027094	0.285323	-0.014001	0.118533	-0.047820	-0.007138	-0.156128	0.012097	1
Geography_Germany	0.005538	0.046897	-0.000567	0.401110	-0.010419	0.010577	-0.020486	0.010297	(
Geography_Spain	0.004780	-0.001685	0.003868	-0.134892	0.009039	-0.013480	0.016732	-0.006482	-(
Gender_Male	-0.002857	-0.027544	0.014733	0.012087	-0.021859	0.005766	0.022544	-0.008112	-(

```
plt.figure(figsize= (10, 10))
sns.heatmap(corr, annot= True, cmap= "coolwarm", linecolor= "black", fmt= "0.2f")
plt.show()
```




```
x = df.drop(columns= ["Exited"], axis= 1)
y = df["Exited"]
```

```
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size= 0.3, random_state= 42)
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
((7000, 11), (3000, 11), (7000,), (3000,))
```

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
# Logistic regression
```

```
lr = LogisticRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
```

```
y_pred.shape
```

```
(3000,)
```

```
# Accuracy
```

```
Accuracy = accuracy_score(y_test, y_pred)
Accuracy
```

```
0.8113333333333334
```

```
# NBC (naive bayes classifier)
```

```
nbc = GaussianNB()
nbc.fit(X_train, y_train)
y_pred_nbc = nbc.predict(X_test)
```

```
Accuracy_nbc = accuracy_score(y_test, y_pred_nbc)
Accuracy_nbc
```

```
0.8323333333333334
```

```
y_pred_t = nbc.predict(X_train)
Acc = accuracy_score(y_train, y_pred_t)
Acc
```

```
0.8214285714285714
```